Contents

I Math **2**

1 Operators **3**
1.1 Linear vs NonLinear 3
1.2 Vector 3
1.3 Backend 3
1.3.1 Gpu 3
1.3.2 Cpu 7

2 Solvers **11**

Part I

Math

Chapter 1

Operators

The operator package contains the classes for matrical operations like vectors, matrix and all the library for hardware acceleration

1.1 Linear vs NonLinear

here is the packages defining the default cpu serial implementations of matrices and polynomial operators

1.2 Vector

Interfaces for Integer and Real vectors. There is no generic Vector class on purpose, it would force the use of the wrapper and an array of Objects isn't contious in memory + it adds a pointer so it makes it 4 bytes heavier per elements. It would also create a lot of

1.3 Backend

where the library for cpu multithreading and gpu acceleration is defined.

1.3.1 Gpu

The GPU backend accelerates the vectors, the padded-CSR matrices and the polynomial tensors of the operator package. It sits behind one abstraction, `GpuExecutor`, with two implementations: `OpenCLGpuExecutor` and `Vulkan GpuExecutor`. Everything that computes is written once and runs on either

of them. Every numerical class exists in a double precision and a single precision flavour with the same public interface.

Building blocks.

`GpuExecutor` One device and one command queue. It allocates typed device buffers (double, float, int, byte), moves data in and out (ranges with offsets, or a single element), copies and fills buffers, creates kernels, and can wait for everything submitted so far (`finish()`). `supports( kernel)` tells whether a kernel can run on the device before it is compiled.

`GpuExecutable` Base class of every object that owns device data (vectors, matrices, tensors). `setExecutor` attaches it to a device and (re)creates its buffers; `bindKernels` binds the kernels the class declares to that executor.

`PortableKernel` A kernel is written twice, once in OpenCL C and once in GLSL compute, together with the capabilities each version needs (an OpenCL extension, a Vulkan feature). It is declared as a static constant in the class that uses it and bound once per executor. `use( globalSize, args...)` enqueues it without waiting; `useBlocking` waits for it.

`GpuResource, ResourceEvent` A `GpuResource` is a kernel argument: a device buffer, or a scalar passed by value. A buffer remembers the last operation that touched it as a `ResourceEvent` (see *Execution and ordering* below) and can be released.

Data classes. Table 1.1 lists the classes.

Padded CSR. A matrix with m rows stores exactly n entries per row (`nnzPerRow`). Entry k of row i has the value v_{in+k} and the column c_{in+k}. A row with fewer than n real entries is completed with padding: value 0 and column i (or 0 when i is outside the matrix). Padding adds nothing to a sum, so a kernel needs no per-row length: one thread handles one row and runs a loop with a fixed trip count,

$$y_i = \sum_{k=0}^{n-1} v_{in+k}\, x_{c_{in+k}}. \tag{1.1}$$

Class	Interface	Holds
`GpuDoubleVector` `GpuFloatVector`	`RealVector`	n real numbers
`GpuIntegerVector`	`IntegerVector`	n 32-bit integers
`GpuBooleanVector`	`BooleanVector`	n flags, one byte each
`GpuPaddedCSRMatrixD` `GpuPaddedCSRMatrixF`	`MutableMatrix`	a sparse matrix with a fixed number of entries per row
`GpuPaddedCSR3TensorD` `GpuPaddedCSR3TensorF`	—	a quadratic form $F_i(x)$
`GpuPaddedCSRTensorD` `GpuPaddedCSRTensorF`	—	a polynomial form $F_i(x)$ of any order

Table 1.1: The GPU classes. The suffix `D` or `F` (and `Double` or `Float` for the vector) names the precision; the integer and boolean vectors have none.

Tensors. The quadratic form (`GpuPaddedCSR3Tensor`) gives each equation i a list of terms t, each with a coefficient c_{it} and two variable indices j_{it}, k_{it}:

$$F_i(x) = \sum_t c_{it}\, x_{j_{it}} x_{k_{it}}, \qquad (J(x)\,\delta)_i = \sum_t c_{it}\,(\delta_{j_{it}}\, x_{k_{it}} + x_{j_{it}}\,\delta_{k_{it}})\,. \tag{1.2}$$

The Jacobian–vector product needs no special case for $j = k$: it then reduces to $2c\,x_j\delta_j$ by itself. `GpuPaddedCSRTensor` generalises this to order N, with $d = N - 1$ variable indices per term:

$$F_i(x) = \sum_t c_{it} \prod_{r=1}^{d} x_{j_{itr}}, \qquad (J(x)\,\delta)_i = \sum_t c_{it} \sum_{m=1}^{d} \delta_{j_{itm}} \prod_{r\neq m} x_{j_{itr}}. \tag{1.3}$$

In both cases one thread evaluates one equation.

Precision. The classes ending in `D` compute in double precision; those ending in `F`, and `GpuFloatVector`, in single precision. The public interface is the same and stays `double`-typed, like `RealVector`: a value given to a float class is rounded to `float` on the way in and widened on the way out, so a call site changes precision by changing a class name. The kernels of the two precisions have different names (the float ones end in `_float`), because the OpenCL profiler aggregates timings by kernel name and a shared name would merge the two.

Double precision is the only thing that needs the optional fp64 capability of a device. A double kernel declares it (`cl_khr_fp64` for OpenCL, `shaderFloat64` for Vulkan), so on a device without it `supports` answers `false` and binding fails cleanly with `UnsupportedOperationException`. The float kernels declare no optional capability and run on every Vulkan device. The OpenCL executor still requires `cl_khr_fp64` when it is created, so on OpenCL the float classes need an fp64-capable device for now.

Transposed product. For $y = A^\top x$ every entry is added to the column it points at,

$$y_c = \sum_{e\,:\,c_e = c} v_e \, x_{\lfloor e/n \rfloor}, \qquad e = in + k, \tag{1.4}$$

and there are two ways to evaluate it.

Scatter One thread per row adds its contributions into y_{c_e}. Two rows can point at the same column, so the addition must be atomic, and core Vulkan and OpenCL have no atomic add on a floating-point buffer (extensions exist, but not on every device): it is emulated with a compare-and-swap loop on the bit pattern. For double precision that needs 64-bit buffer atomics (`cl_khr_int64_base_atomics`, `shaderBufferInt64 Atomics`). The result must be cleared first, and the summation order changes from run to run.

Gather One thread per output column sums the entries that point at it, using an index $E(c) = \{e : c_e = c\}$ stored CSR-style over the flat entry numbers. It needs no atomics and no clearing, and it always sums in the same order, so its results are reproducible.

The float matrix always gathers. The double matrix scatters when the device has 64-bit atomics and gathers when it does not. The index is built on the host by a counting sort in $O(mn)$, uploaded once and cached. Only a change of the column pattern invalidates it (a `setRow` that changes a column, `resize`, a new executor). `set` and `add` change values only, and the kernel reads values through the flat entry number, so the cached index stays valid.

Execution and ordering. Dispatches, copies, fills and (on Vulkan) uploads return as soon as the work is submitted. Only a download, `use Blocking` and `finish()` wait (on OpenCL an upload does too, see *Limits*). Correctness rests on one rule: *an operation that touches a buffer is ordered after the last operation that touched that buffer, and nothing else is ordered.*

A vector that is uploaded and then used in a matrix product does not hold up unrelated work, and the product starts only once its inputs are ready. The rule covers read-after-write, write-after-read and write-after-write alike. A buffer does not record whether its last access was a read or a write, so two reads of the same buffer are ordered as well; that is conservative, never wrong.

	OpenCL	Vulkan
Queue	out-of-order, with profiling	one in-order queue; every operation is its own command buffer with its own fence
Event	a `cl_event`	the serial number of a submission (`VulkanEvent`)
Dependency	the pending events of the arguments form the event wait list of the enqueue, so it is precise per buffer	a memory barrier opens the command buffer, only if a touched buffer has unfinished work (below)
Upload	blocking write	non-blocking, through a pool of staging buffers
Download	blocking read, after the buffer's events	waits for its own fence only, outside the executor lock
Release	the OpenCL runtime keeps a buffer alive until the queued commands that use it finish	destruction is queued behind the buffer's last serial

Table 1.2: How the two backends implement the ordering rule.

Vulkan: serials and barriers. Every submission receives the next value of a counter, its *serial*. Finished submissions are retired oldest first, which yields a watermark c: every submission up to c has completed. A buffer r remembers the serial p_r of the last submission that touched it ($p_r = 0$ if none), and the executor remembers b, the serial of the latest submission that opened with a barrier. A new submission that touches the set of buffers T opens with a global memory barrier if and only if

$$\exists\, r \in T : \quad p_r > c \;\; \wedge \;\; p_r \geq b. \tag{1.5}$$

The first condition says the earlier work may still be running. The second says no later barrier already orders it, since a barrier at b makes every submission before b finish first. When no touched buffer qualifies, no barrier is recorded and the submission may overlap with whatever else is in flight (figure 1.1). Afterwards the new serial becomes p_r of every buffer in T. Only the newest serial is kept: a barrier orders everything submitted before it, so waiting for the newest covers all older ones.

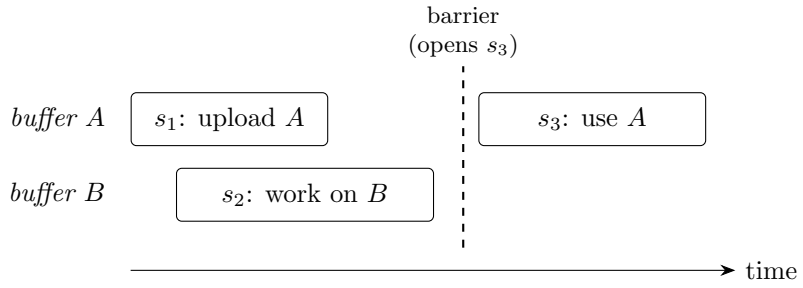


Figure 1.1: s_1 and s_2 touch different buffers: no barrier, they overlap. s_3 uses A, whose last operation s_1 may still be running, so it opens with a barrier. A barrier cannot name a single buffer, so s_3 also waits for s_2, which it does not depend on.

Vulkan: transfers. An upload copies the host data into a staging buffer taken from a pool, submits the device copy and returns, so the caller's array can be reused at once. The staging block returns to the pool when its submission completes. A block holds at least 1 MiB and at most 64 MiB (a larger transfer is split into chunks), and at most 256 MiB of staging memory exists at any time. Beyond that an upload waits for earlier ones to drain instead of allocating more; this is the only case where an upload blocks. A download submits a copy into a staging block and waits for that submission's fence alone. Everything it depends on is ordered before it on the device, so it does not wait for unrelated work.

Vulkan: lifetime and threads. A buffer cannot be freed while queued work still uses it, so `release()` queues the destruction behind the buffer's own serial p_r and carries it out once the watermark reaches it, at once if the buffer is idle. On OpenCL the runtime does this by reference counting. Submission and bookkeeping run under one monitor of the executor and host waits run outside it, so other threads keep submitting while one waits for a

download. Each buffer also has a small monitor of its own; the lock order is always executor first, buffer second. Scalar arguments travel in a 128-byte push-constant block, a kernel has at most 8 buffer bindings, and the pool holds 4096 descriptor sets, recycled as their submissions retire; when it runs dry the executor waits for in-flight work to free some.

Limits. A Vulkan barrier cannot name a buffer: it orders all earlier work, so a dependent operation also waits for unrelated earlier work (figure 1.1). OpenCL, with an out-of-order queue and event wait lists, is precise here. There is one Vulkan queue; using several would require a serial per queue and semaphores for the dependencies between them. `pendingEvents()` on a Vulkan buffer reflects the last retirement pass, so it can lag the device slightly but never run ahead of it. On OpenCL, host-to-device writes are blocking.

Errors. Launches are validated before anything is submitted: `globalSize` must be a multiple of `localSize`, the number of buffer arguments must match the kernel, and a `null` or foreign resource is refused. Submitting work to an executor after `close()` throws `IllegalStateException`; closing twice and releasing a buffer twice are harmless.

Tests. The behaviour contract (`GpuContractTest`) is written once and runs for both backends and both precisions. Each result is compared with the CPU implementation in double precision; for the float classes the CPU side is given the float-rounded inputs, so the comparison measures the arithmetic of the GPU and not the loss of precision of the input. Table 1.3 gives the tolerances. The double flavour is skipped, not failed, on a device without fp64; the float flavour always runs. Three more groups complete the suite.

Comparison	Double	Float
Element-wise result, relative to $\max(1, \lvert y_i \rvert)$	10^{-12}	$2 \cdot 10^{-4}$
Reduction (dot product, norm), relative	10^{-10}	10^{-4}

Table 1.3: Tolerances of the GPU tests against the CPU reference.

A test that needs no GPU compiles every kernel to SPIR-V, checks that kernel names are unique, that each OpenCL entry point carries its kernel's name, that a kernel declares fp64 exactly when it uses `double`, and that no

float kernel does. The Vulkan executor tests make the host race the GPU on purpose: upload then use, read then overwrite, and overlapping writes must all give the result of the program order, and several threads share one executor. Running with `-Dformicapi.test.requireGpu=true` turns an unavailable device from a skip into a failure, for a machine that is supposed to have one.

1.3.2 Cpu

The CPU backend provides the reference implementation of the operator interfaces and an optional shared execution service for operations that benefit from data parallelism. Its design is deliberately simpler than the GPU backend: numerical data remains in ordinary Java primitive arrays, and a computation is expressed as a serial loop or as a task over contiguous index ranges. No device transfer or kernel compilation is involved.

Architecture and ownership. `CpuExecutable` is the common base class for the CPU data structures. It contains an optional, nullable reference to a `CpuExecutor`, set through `setExecutor`. An object without an executor runs its supported operations serially. Attaching an executor does not move or copy its data: the vectors, matrices and tensors continue to own their primitive arrays on the host. Objects can share one executor, which is generally preferable to creating a separate worker set for every object.

`CpuExecutor` implements `BackendExecutor` and `AutoCloseable`. It owns a fixed number of worker threads and one dispatcher thread. The dispatcher takes submitted jobs from a concurrent queue in submission order; workers execute the current job's assigned range. Calls are synchronous from the caller's point of view: the calling thread waits until the job has completed and, for reductions or accumulations, until its result has been combined. The computation itself runs on the workers. Multiple caller threads may share an executor, but their jobs are serialized by the single dispatcher rather than dispatched concurrently as independent jobs.

Job model and range partitioning. Every dispatch is represented by a job. A job defines the work performed by a worker over a half-open interval $[start, end)$ and the combination step, if any, performed after all workers finish. For a problem of size N and P workers, the executor uses a fixed contiguous partition with chunk size

$$C = \left\lceil \frac{N}{P} \right\rceil, \qquad I_p = [pC,\, \min(N, (p+1)C)). \tag{1.6}$$

Empty ranges are skipped. This partition is simple and gives each worker one contiguous region, which is suitable for the array-oriented loops used throughout the backend. It is not a work-stealing scheduler: a worker that finishes its range does not take work from another worker.

The executor exposes four task forms:

`parallelFor(size, task)` Runs a range task for each non-empty worker interval. It is intended for independent writes, such as applying an element-wise vector operation or evaluating separate matrix rows.

`parallelReduceDouble(size, task)` Runs a double-valued partial reduction per worker, then adds the partial results on the dispatcher thread.

`parallelReduceFloat(size, task)` The corresponding float reduction, with float partial results and a float final sum.

`parallelForAccumulate(size, output, task)` Gives each worker a private accumulation array and combines those arrays into the output after the parallel phase. This is intended for scatter operations where different input elements can contribute to the same output index.

The reduction methods return zero for a non-positive size, and the range and accumulation methods do no work for a non-positive size. The final reduction is a sum of the per-worker partial values. Since the grouping of floating-point additions differs from a serial left-to-right loop, a parallel reduction can differ slightly in its last bits even when the mathematical operation is unchanged.

Worker lifecycle and dispatch overhead. Workers are persistent for the lifetime of the executor; dispatch does not create new threads. To reduce latency between successive jobs, the dispatcher, workers and waiting callers briefly busy-spin, then park when idle. The default spin interval is `DEFAULT_SPIN_NANOS` ($20\,\mu$s) and can be changed with `setSpinNanos`. This trades a small amount of CPU activity during short idle periods for lower wake-up latency. Setting the interval to zero parks immediately.

Parallel execution is not automatically beneficial. The executor exposes `DEFAULT_PARALLEL_THRESHOLD` (4096), `getParallelThreshold()`, `setParallelThreshold(...)` and `shouldUseParallel(size)`. The threshold is a workload heuristic, not a hardware guarantee: the crossover depends on processor, memory bandwidth, worker count and the cost of the operation. The static `calibrateThreshold(threads)` method measures serial and

parallel range loops over a set of candidate sizes and returns the first candidate at which parallel execution wins (or the largest candidate if it never wins during the sweep). The overload accepting candidate sizes allows a caller to choose the sweep. Calibration is best treated as a machine- and workload-specific starting point, not a permanent universal optimum. An executor is created with a worker count, `new CpuExecutor(threads)`, or with a worker count and a threshold, `new CpuExecutor(threads, threshold)`; the tests use a very small threshold so that tiny systems still take the parallel path.

The executor's `parallelFor` methods always perform a dispatch; the decision to stay serial is made by the data structure before calling them. Most vector operations use `shouldUseParallel`, while the padded matrix and tensor implementations compare their row/equation count with `getParallelThreshold()` directly. Code using the executor should therefore apply the threshold at the call site rather than assume that every executor method silently falls back to a serial loop.

Avoiding write races: accumulation. A range task is safe when workers write to disjoint output locations. A transpose sparse-matrix product is different: several matrix rows can contain entries for the same output column, so parallel row processing would race if every worker updated the shared result array directly. `parallelForAccumulate` avoids atomics by allocating one private double array per worker. Each worker writes only to its own array during the parallel phase. Once all workers finish, a second parallel range job folds the private arrays into the caller's output and resets the scratch entries to zero. The scratch arrays are retained and reused across calls when the output length is unchanged. This removes contention during the scatter phase, at the cost of $O(PM)$ scratch storage and clearing/folding work for an output of length M, even if relatively few entries are touched.

CPU data structures. The concrete implementations present in the backend are summarised in table 1.4.

The vector classes use primitive arrays rather than boxed values. The double and float real-vector implementations contain parallel paths for element-wise arithmetic, indexed operations, dot products and reductions; the float implementation mirrors the double implementation while retaining float storage and float partial sums. Integer-vector operations are currently serial. Boolean-vector operations are also mostly serial, with parallel indexed gather when an executor is attached and the input size crosses

Class	Interface	Storage and role
`CpuDoubleVector`	`RealVector`	Double-precision primitive array; vector arithmetic and reductions.
`CpuFloatVector`	`RealVector`	Float-precision primitive array with the real-vector interface.
`CpuIntegerVector`	`IntegerVector`	32-bit integer primitive array.
`CpuBooleanVector`	`BooleanVector`	Boolean primitive array.
`CpuPaddedCSRMatrixD`	`MutableMatrix`	Double-precision sparse matrix with a fixed number of storage slots per row.
`CpuPaddedCSR3TensorD`	`GeneralOperator`	Fixed-capacity quadratic polynomial operator.
`CpuPaddedCSRTensorD`	—	Fixed-capacity polynomial operator of configurable order.

Table 1.4: CPU backend implementations present in the source tree.

the threshold. The implementation deliberately leaves operations such as `System.arraycopy` and `Arrays.fill` serial: these are already highly optimised JVM intrinsics and splitting them among workers is not expected to improve their throughput.

Sparse matrix execution. `CpuPaddedCSRMatrixD` stores a fixed number q of slots per row in flat value and column-index arrays. For row i, slot k is stored at $iq + k$; unused slots are zero-valued. Matrix-vector multiplication assigns complete rows to workers, so each worker writes distinct result entries and no synchronisation is needed. The transposed product instead uses the private-buffer accumulation described above, because multiple rows may contribute to the same output column. The current CPU matrix implementation is double precision; the source tree also contains standalone padded CSR matrix representations outside this backend.

Polynomial tensor execution. `CpuPaddedCSR3TensorD` stores a fixed number of quadratic terms per equation. Each term contains a coefficient and two variable indices. `CpuPaddedCSRTensorD` generalises the representation to a configurable polynomial order, storing the variable indices for each term. Evaluation assigns equations to workers, so each worker writes a dis-

joint range of result entries. The Jacobian–vector product follows the same equation-wise partition: the input vector and direction are read-only, and each equation's output is written once. As with the matrix, small equation counts use a serial range loop and larger ones use the attached executor.

Numerical and memory considerations. The CPU backend is the reference path against which accelerated implementations can be compared, but parallel execution is not necessarily bitwise identical to serial execution for reductions because floating-point addition is not associative. Range-based element-wise operations preserve the per-element arithmetic order. The accumulation strategy also has a defined two-phase structure, although the order in which worker-local sums are folded differs from a serial scatter order.

Vector constructors that accept primitive arrays retain the supplied array rather than making an unconditional defensive copy, and the `array()` accessors expose the backing storage. Callers should therefore treat those arrays as shared mutable state: external mutation is visible to the vector, and concurrent mutation while an operation is running is not made safe by the executor. The executor synchronises job dispatch and completion, not arbitrary access to data structures from application threads.

Lifecycle and typical use. The executor owns long-lived worker and dispatcher threads and should be closed when the application no longer needs it. It implements `AutoCloseable`, so try-with-resources is the preferred pattern (the benchmarks also call `shutdown()` directly). Shutdown is intended after callers have finished submitting work; it is not designed as a cancellation mechanism for concurrently active clients. Data structures do not own the executor and may share it, so the code that creates the executor should also own its lifetime.

```
try (CpuExecutor executor = new CpuExecutor(Runtime.getRuntime().
    availableProcessors())) {
    CpuDoubleVector x = new CpuDoubleVector(values);
    CpuDoubleVector y = new CpuDoubleVector(values.length);
    x.setExecutor(executor);
    y.setExecutor(executor);

    x.axpy(2.0, y);           // serial or parallel according to the
     threshold
    double product = x.dot(y);
}
```

A single executor can be attached to several vectors and operators. The caller remains responsible for ensuring that the objects and their backing arrays are not mutated concurrently in ways that conflict with the operation being executed.

Comparison with the GPU backend. Both backends implement the same interfaces, so a solver does not know which one it runs on; table **??** lists what differs underneath.

	CPU	GPU
Data	primitive arrays on the host	device buffers, with a host mirror in the matrices and tensors
Execution	synchronous: a call returns when the result is ready	asynchronous: a call returns once the work is submitted, ordered per buffer
Transposed product	private accumulation array per worker, folded afterwards	atomic scatter, or a gather through a cached column index
Reproducibility	row-wise products are bit-identical to the serial loop; sums over several workers can differ in the last bits	the gather is reproducible; the atomic scatter is not
Precision	double, with float vectors	double and float classes with the same interface

Table 1.5: The CPU and GPU backends side by side.

Tests. `ParallelCpuTest` runs the same random 7-point-stencil matrix (64^3 rows) through the serial reference `PaddedCSRMatrixD` and through `Cpu PaddedCSRMatrixD` on a four-worker executor. The matrix–vector product is compared with a tolerance of exactly zero: each row is summed in the same order, so the parallel result must be bit-identical. The transposed product is compared with a tolerance of 10^{-9} because the order of the accumulation differs; that test is currently commented out and should be re-enabled. The CPU classes also serve as the reference in the GPU contract tests, where

the executor is created with two workers and a threshold of 16 so that small systems exercise the parallel path.

Chapter 2

Solvers

The solvers turn the operators of the previous chapter into answers. They are matrix-free: a solver never looks inside a matrix, it only asks for products (Ax, and $A^\top x$ for least squares), so the same algorithm runs on any operator that provides them. The vector-based entry points are written against the `Matrix` and `RealVector` interfaces. The CPU and GPU benchmarks call the same `LeastSquare.solve` with CPU and with GPU objects. Array-based entry points take and return `double[]` and are meant for small systems and tests.

Table **??** gives an overview. All iterative solvers take an iteration limit and a tolerance, and are warm-started: the initial guess is the vector given in, and a system that is already solved returns at once.

Solver	Solves	Requires
`Conjugate Gradient`	$Ax = b$	A symmetric positive definite
`BiCGStab`	$Ax = b$, also with fixed unknowns	A square, any structure
`LeastSquare`	$\min_x \lVert Ax - b \rVert_2$	any shape, needs A and $A^\top$ products
`NewtonKrylov`	$C(x) + Ax = b$	quadratic term C with a Jacobian–vector product
`SimpleSolvers`	$\min_x f(x)$, one variable	a scalar function

Table 2.1: The solver package.

Working buffers. The vector-based solvers allocate nothing per iteration. The caller hands them a `WorkingBuffer`, a fixed set of scratch vectors, built either from an array of vectors or from a count and a factory, for instance `new WorkingBuffer<>(7, () -> new CpuDoubleVector(n))`. Because the caller creates the vectors, they belong to the backend of the caller's choice: CPU vectors with an executor attached, or GPU vectors.

2.1 Conjugate gradient

For a symmetric positive definite matrix, `ConjugateGradient` computes the iterate $x_k \in x_0 + \mathcal{K}_k(A, r_0)$, with $r_0 = b - Ax_0$ and $\mathcal{K}_k(A, r_0) = \operatorname{span}\{r_0, Ar_0, \ldots, A^{k-1}r_0\}$, that minimises the error $\|x_\star - x_k\|_A$. It does so with the short recurrence

$$\begin{aligned} \alpha_k &= \frac{r_k^\top r_k}{p_k^\top A p_k}, \qquad x_{k+1} = x_k + \alpha_k p_k, \qquad r_{k+1} = r_k - \alpha_k A p_k, \\ \beta_k &= \frac{r_{k+1}^\top r_{k+1}}{r_k^\top r_k}, \quad p_{k+1} = r_{k+1} + \beta_k p_k, \end{aligned} \tag{2.1}$$

with $p_0 = r_0$: one product with A, two dot products and three vector updates per iteration, and in exact arithmetic termination after at most n steps. The entry point is `ConjugateGradient.solve(A, x0, b, maxIter, tol)`, which returns the solution as a new array. It must only be used with a symmetric positive definite matrix; on other matrices it can stall or diverge.

2.2 BiCGStab

For a square matrix without symmetry, `BiCGStab` is the stabilised bi-conjugate gradient method. It needs two products with A per iteration and none with $A^\top$, which makes it usable with operators that can only be applied forwards. The entry points are `BiCGStab.solve(A, b, maxIter, tol)`, which starts from zero, and `BiCGStab.solve(A, x0, b, maxIter, tol)`, which starts from the given guess.

Fixed unknowns. `BiCGStab.solveConstrained(A, x, fixed, b, maxIter, tol)` solves for the unknowns with `fixed[i] == false` and keeps the others at the values in `x`. The columns of the fixed unknowns are moved to the right-hand side,

$$A_{:,F}\, x_F = b - A_{:,D}\, x_D, \tag{2.2}$$

where F is the set of free unknowns and D the set of fixed ones; the reduced system must be square. For $A = \begin{pmatrix} 2 & 1 & 0 \\ 1 & 3 & 4 \end{pmatrix}$, $b = (4, 13)^\top$ and $x_2 = 2$ held fixed, the free unknowns are $x_0 = 1.4$ and $x_1 = 1.2$. This is how prescribed values (boundary conditions) are imposed without changing the matrix.

2.3 Least squares

`LeastSquare` minimises $\|Ax - b\|_2$ for a matrix of any shape: square, overdetermined (the least-squares solution when no exact solution exists) or underdetermined. The method is LSQR of Paige and Saunders, built on the Golub–Kahan bidiagonalisation

$$\beta_{k+1} u_{k+1} = A v_k - \alpha_k u_k, \qquad \alpha_{k+1} v_{k+1} = A^\top u_{k+1} - \beta_{k+1} v_k, \tag{2.3}$$

with $\beta_1 u_1 = b$ and $\alpha_1 v_1 = A^\top u_1$. It is mathematically equivalent to conjugate gradients on the normal equations $A^\top A x = A^\top b$, but it never forms $A^\top A$, which would square the condition number. Each iteration costs one product with A and one with $A^\top$, which is why the matrix interface offers `transposeApply` and why that operation is optimised on both backends. The working buffers hold two vectors of the length of b and three of the length of x.

`LeastSquare.solve(A, b, x, maxIter, tol)` exists for arrays and for vectors, and the vector form with two working buffers returns the number of iterations used. The vector x is the initial guess on entry and the solution on exit. A right-hand side or a guess of the wrong length raises `IllegalArgumentException`. A zero right-hand side gives the zero solution, and an initial guess that is already the solution is accepted without iterating.

2.4 Newton–Krylov

`NewtonKrylov` solves the nonlinear system

$$F(x) = C(x) + Ax - b = 0, \qquad C_i(x) = \sum_t c_{it}\, x_{j_{it}} x_{k_{it}}, \tag{2.4}$$

where C is a quadratic form stored in a padded-CSR tensor (the *Tensors* paragraph of the Gpu part) and A is any matrix. Each Newton step $x \leftarrow x + \delta$ requires the linear system

$$J(x)\,\delta = -F(x), \qquad J(x)\,\delta = C'(x)\,\delta + A\,\delta, \tag{2.5}$$

and the Jacobian is never formed. Only its action on a vector is needed, and the tensor classes provide the exact Jacobian–vector product $C'(x)\delta$ for this purpose. The step is computed only approximately by a Krylov method, which is what makes the method an inexact Newton method. Near a root where J is nonsingular, Newton's method converges quadratically; the inexactness of the inner solve lowers the rate.

The call is `NewtonKrylov.solve(C, A, x, b, n1, n2, t1, t2, null, null, null, buffer7, buffer3)`. Here x is both the initial guess and the result, two iteration limits and two tolerances control the nonlinear and the linear iteration, and the two working buffers hold 7 and 3 vectors. **[TODO: say which limit and tolerance belongs to which iteration, which Krylov method solves the step, whether a line search is used, and what the three optional arguments (null in every test) are]**

The tests solve $x_0^2 + x_1 = 3$, $x_1^2 + x_0 = 5$ (root $(1, 2)$), a coupled quadratic system started from $(0.1, 0.1)$, and a quadratic term with a nonzero linear part started far from its root, at $(5, 5)$, each to 10^{-8}. A nonlinear system can have several roots; the one found depends on the initial guess.

2.5 Scalar minimisation

`SimpleSolvers.gradientDecent` minimises a function of one real variable by gradient descent, in single precision. It takes the function, a starting point and two float parameters (the tests use 0.1 and 0.01), and returns the minimiser. The tests minimise x^2 from -1 and $|\cos x|$ from 10^{-3}, whose minimum $\pi/2$ is a kink where the derivative does not exist, each to 10^{-6}. **[TODO: name the two float parameters and the stopping rule, and note that the method name is spelled `gradientDecent` in the code]**

2.6 Choosing a solver

For a symmetric positive definite system use `ConjugateGradient`: it is the cheapest, with one product per iteration. For a square system without symmetry use `BiCGStab`. When the matrix is rectangular, singular, or the system has no exact solution, use `LeastSquare`; it is also the safe choice when nothing is known about the matrix, at the price of a transposed product per iteration. For the nonlinear system $C(x) + Ax = b$ use `NewtonKrylov`.

Tolerances and stopping. **[TODO: state whether the tolerance is absolute or relative to $\|b\|$, which norm is tested, and what is returned when the iteration limit is reached without convergence]**

Tests. The solvers are tested against systems with known answers. `ConjugateGradient` on a symmetric positive definite 3×3 matrix. `BiCGStab` on a 2×2 and two nonsymmetric 3×3 systems, the identity, an already-solved warm start, a constrained solve, and an ill-conditioned diagonal matrix with entries from 10^{-6} to 10^{6} (accuracy 10^{-6}). `LeastSquare` on identity, diagonal, symmetric, general square, consistent and inconsistent overdetermined and Hilbert systems; for the inconsistent system it checks optimality by perturbing the solution in several directions and verifying that the residual never decreases. `NewtonKrylov` on the three quadratic systems above.